Date:22/10/2026

CO2-AT1 – ANALYTICAL PROBLEM SOLVING

Course Code: DSA0308

Course name: Natural Language Processing

Reg No: 192324315L **Name:** A. Gayathri Shanmukhi

1. Program

```
words = ["connected", "connecting", "connection"]

print(f"{'Word':<15}{'Root':<12}{'Suffix':<12}{'Type':<18}{'Normalized'}")

for word in words:

    if word == "connected":

        root, suffix, typ = "connect", "ed", "Inflectional"

    elif word == "connecting":

        root, suffix, typ = "connect", "ing", "Inflectional"

    else:

        root, suffix, typ = "connect", "ion", "Derivational"

    print(f"{word:<15}{root:<12}{suffix:<12}{typ:<18}{root}")
```

Output

```
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================ RESTART: C:/Users/shanm/OneDrive/Desktop/exp1.py ================
Word           Root        Suffix      Type              Normalized
connected      connect     ed          Inflectional      connect
connecting     connect     ing         Inflectional      connect
connection     connect     ion         Derivational      connect
>>>
```

2.program

```
words = ["unhappy", "happiness", "happily"]

print(f"{'Word':<15}{'Prefix':<10}{'Root':<10}{'Suffix':<10}{'Type':<18}{'Normalized'}")

for word in words:

    if word == "unhappy":

        prefix, root, suffix = "un", "happy", "-"

    elif word == "happiness":

        prefix, root, suffix = "-", "happy", "ness"

    else:

        prefix, root, suffix = "-", "happy", "ly"

    print(f"{word:<15}{prefix:<10}{root:<10}{suffix:<10}{'Derivational':<18}{root}")
```

Output

```
========== RESTART: C:/Users/shanm/OneDrive/Desktop/exp1.py ==========
Word            Prefix    Root      Suffix    Type              Normalized
unhappy         un        happy     -         Derivational      happy
happiness       -         happy     ness      Derivational      happy
happily         -         happy     ly        Derivational      happy
>>>
```

3.program

```
words = ["played", "player", "playing"]

print(f"{'Word':<12}{'Stem':<12}{'Removed Affix':<16}{'Type':<18}{'Normalized'}")

for word in words:

    if word == "played":

        stem, affix, typ = "play", "ed", "Inflectional"

    elif word == "player":

        stem, affix, typ = "play", "er", "Derivational"

    else:

        stem, affix, typ = "play", "ing", "Inflectional"

    print(f"{word:<12}{stem:<12}{affix:<16}{typ:<18}{stem}")
```

Output

```
========== RESTART: C:/Users/shanm/OneDrive/Desktop/exp1.py ==========
Word        Stem        Removed Affix   Type              Normalized
played      play        ed              Inflectional      play
player      play        er              Derivational      play
playing     play        ing             Inflectional      play
>>>
```

4. Program

```
words = ["writes", "writing", "written"]

rules = {

    "writes": ("write", "s", "Regular"),

    "writing": ("write", "ing", "Regular"),

    "written": ("write", "ten", "Irregular")

}

print(f"{'Word':<12}{'State Path':<28}{'Root':<10}{'Suffix':<10}{'Type':<12}{'Normalized'}")

for word in words:
```

```
    root, suffix, typ = rules[word]
    path = "Start -> Identify -> Root -> End"
    print(f"{word:<12}{path:<28}{root:<10}{suffix:<10}{typ:<12}{root}")
```

Output

```
>>
================ RESTART: C:/Users/shanm/OneDrive/Desktop/exp1.py ================
Word        State Path                  Root      Suffix    Type         Normalized
writes      Start -> Identify -> Root -> Endwrite     s         Regular     write
writing     Start -> Identify -> Root -> Endwrite     ing       Regular     write
written     Start -> Identify -> Root -> Endwrite     ten       Irregular   write
>>
```

5.program

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
words = ["relational", "relation", "relate"]
print(f"{'Word':<15}{'Step 1':<15}{'Step 2':<15}{'Final Stem'}")
for word in words:
    step1 = word
    step2 = word
    if word.endswith("ational"):
        step1 = word.replace("ational", "ate")
    elif word.endswith("tional"):
        step1 = word.replace("tional", "tion")
    step2 = ps.stem(step1)
    final_stem = ps.stem(word)
    print(f"{word:<15}{step1:<15}{step2:<15}{final_stem}")
```

Output

```
>>>
================================================================
Word           Step 1         Step 2         Final Stem
relational     relate         relat          relat
relation       relation       relat          relat
relate         relate         relat          relat
>>>
```